作业 HW2* 实验报告

姓名：李欣洋 学号：2453359 日期：2025 年 10 月 13 日

实验报告格式要求按照模板（使用 Markdown 等也请保证报告内包含模板中的要素）
对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考 vscode 配色）
实验报告要求在文字简洁的同时将内容表示清楚
报告内不要大段贴代码，尽量控制在 20 页以内

1. 涉及数据结构和相关背景

题目或实验涉及数据结构的相关背景
实验整体使用到的数据结构包括栈和队列
题目一:给定一个固定的火车序列,使用栈来模拟火车可能的出站序列,并且对此进行判断
题目二:用栈来模拟布尔运算,其中使用到了一个符号栈和一个数据栈
题目三:给定一个字符串,其中的内容由“(”和“)”组成,用栈来求出其中的最长子串以及子串的起始位置
题目四:求解矩阵的联通问题,这类问题在算法题库中一般被称为岛屿连接问题,具体的实现通过BFS(广度优先搜索算法)加上队列这个数据结构来进行实现
题目五:重新设计一个队列的数据结构,要求能输入入队,出队,查找最大值的功能,其中对查找最大值的时间复杂度给出要求,要求时间复杂度尽可能低,控制在 1 秒以内

2. 实验内容

2.1 列车进站

2.1.1 问题描述

每一时刻，列车可以从入口进车站或直接从入口进入出口，再或者从车站进入出口。即每一时刻可以有一辆车沿着箭头 a 或 b 或 c 的方向行驶。 现在有一些车在入口处等待，给出该序列，然后给你多组出站序列，请你判断是否能够通过上述的方式从出口出来。

2.1.2 基本要求

简短地描述题目要求
输入:
第 1 行，一个串，入站序列。
后面多行，每行一个串，表示出栈序列
当输入=EOF 时结束
输出:
多行，若给定的出栈序列可以得到，输出 yes,否则输出 no。

2.1.3 数据结构设计

```
struct Stack
{
    vector<char>data;//作为车站 用于存放已经进站的列车
    int top;//栈顶指针
    int maxsize;
    Stack()
    {
        top=-1;
        maxsize=0;
    };
};
```

2.1.4 功能说明（函数、类）

```
bool Train(Stack &s,vector<char>input,vector<char>output)
{
  //此处展示的是伪代码,仅展现重要的实现逻辑
  //input 代表进站序列,output 代表出站序列
    while(i<input.size())
    {
        if(s.top==-1 || s.data[s.top]!=output[j])//车站为空 或 站顶车次不等于出站车次
        {
            if(i==input.size())
              return false;//车站进车完毕 且车站顶的车次不等于出站车次
            s.data.push_back(input[i]);//进站
        }
        else//车站顶的车次等于出站车次
        {
            s.data.pop_back();//出站
        }
    }
    while(s.top!=-1)
    {
        if(s.data[s.top]!=output[j])
          return false;
        s.data.pop_back();//车站顶的车次等于出站车次 出站
    }
    return true;//全部出站返回true
}
```

2.1.5 调试分析（遇到的问题和解决方法）

此题只经过了一次调试就成功得到了正确的结果了,此题的难点不在于数据结构的设计,因为先进站的列车要在后面才能够出站,这样先进后出的特点刚好符合栈的特性,所以设计一个栈来实现这个题目并不困难.

个人在此题遇到的问题就是算法的设计,该怎么在已经给定一个列车序列的情况下,来判断出站的序列能否实现.思考一下就可以得出,考虑使用两个 vector 数组来存储开始的列车序列和出站序列,然后将开始的列车序列入栈,如果栈顶元素等于出站序列目前的头元素,那么就出栈,如果不等于,那么就继续入栈,在全部入栈之后,再执行上面的判断逻辑,一旦遇到不相等的情况,那么就说明该序列不可行,直接返回 false,若代码能全部执行完,则说明可以,返回 true

2.1.6 总结和体会

通过栈结构模拟进站和出站过程，理解了栈的“先进后出”特性与实际问题的结合。
难点:如何想到使用两个数组来配合栈解决这个问题
易错点:对于 output 数组和栈来说,每一次循环都应该清空之后再执行后续代码,在实际些代码时容易忘记清空栈或输出序列，导致多组数据时结果错误

2.2 布尔表达式

2.2.1 问题描述

计算如下布尔表达式（V | V) & F & (F | V) 其中 V 表示 True，F 表示 False，|表示 or，&表示 and，！ 表示 not（运算符优先级 not> and > or）

2.2.2 基本要求

输入:

文件输入，有若干（A<=20）个表达式，其中每一行为一个表达式。 表达式有（N<=100）个符号，符号间可以用任意空格分开，或者没有空格，所以表达式的总长度，即字符的个数，是未知的。
对于 20%的数据，有 A<=5，N<=20，且表达式中包含 V、F、&、|
对于 40%的数据，有 A<=10，N<=50，且表达式中包含 V、F、&、|、！
对于 100%的数据，有 A<=20，N<=100，且表达式中包含 V、F、&、|、!、(、)
所有测试数据中都可能穿插空格

输出:

对测试用例中的每个表达式输出“Expression”，后面跟着序列号和“: ”，然后是相应的测试表达式的结果（V 或 F），每个表达式结果占一行（注意冒号后面有空格）。

2.2.3 数据结构设计

```
//全局变量计数器
int count_sum=0;
//字符栈
struct Stack_num
{
    vector<char>data;
    int top;
    Stack_num()
    {
        top=-1;
    };
};

//符号栈 符号优先级 !>&>|
struct Stack_op
{
    vector<char>data;
    int top;
    Stack_op()
    {
        top=-1;
    };
};
```

2.2.4 功能说明（函数、类）

```
void Bollexpression(string input,Stack_num &s_num,Stack_op &s_op)
{
  //此处使用伪代码来展示具体的逻辑
    //先清空两个栈
    for(char ch: input)
    {
        if(ch==' ')continue;
        if(ch=='F'||ch=='V')
            s_num.data.push_back(ch);
        //为左括号和!的优先级最高 所以直接入栈即可
        if(ch=='(')
            s_op.data.push_back(ch);
        if(ch=='!')
```

```
            s_op.data.push_back(ch);
            s_op.top++;
        //处理右括号 当遇到右括号时 就一直取出栈顶的运算符进行运算 直至遇到左括号
        //具体的运算过程根据取出的运算符进行
        if(ch==')')
        {
            while(s_op.top!=-1&&s_op.data[s_op.top]!='(')
            {
                char op=s_op.data[s_op.top];//拿到栈顶的运算符
                if(op=='!')
                else if(op=='&')
                else if(op=='|')
            }
            //把左括号弹出
            if(s_op.top!=-1&&s_op.data[s_op.top]=='(')
        }
        else if(ch=='&'||ch=='|')
        {
            while(s_op.top!=-1&&(s_op.data[s_op.top]=='!'||(s_op.data[s_op.top]=='&'&&ch=='|')))
            {
                char op=s_op.data[s_op.top];//拿到栈顶的运算符
                if(op=='!')
                else if(op=='&')
                else if(op=='|')
            }
            //把当前符号入栈
            s_op.data.push_back(ch);
        }
    }
    //把栈中剩余的符号依次出栈
    while(s_op.top!=-1)
```

2.2.5 调试分析（遇到的问题和解决方法）

括号匹配问题:
在实际的运行过程中,如果表达式括号不匹配，可能导致符号栈无法正确弹出，出现越界或死循环。
输入格式问题:
因为在测试过程中的输入案例中有多余空格，如果未做处理则会导致错误读入空格作为运算字符,从而导致程序运行错误.

2.2.6 总结和体会

通过双栈法处理布尔表达式，掌握了运算符优先级和括号处理的技巧,同时,根据高程中学习到的前置知识,这样的思路还可以用来处理我们常用的四则运算,四则运算也遵循布尔代数的运算法则.

难点:对括号的处理,如何对左右括号进行特殊处理

易错点:优先级判断和弹栈条件容易写错，导致结果不正确

2.3 最长子串

2.3.1 问题描述

已知一个长度为 n，仅含有字符'('和')'的字符串，请计算出最长的正确的括号子串的长度及起始位置，若存在多个，取第一个的起始位置。
子串是指任意长度的连续的字符序列。
例 1：对字符串 "((()))()"来说，最长的子串是"((()))"，所以长度=6，起始位置是 0。
例 2：对字符串")())"来说，最长的子串是"()",子串长度=2，起始位置是 1。
例 3；对字符串""来说，最长的子串是"",子串长度=0，空串的起始位置规定输出 0。

2.3.2 基本要求

输入: 一行字符串。
输出: 子串长度，及起始位置

2.3.3 数据结构设计

```
struct Stack
{
    vector<char>data;//存储输入的括号串
    int top;//栈顶指针
    Stack()
    {
        top=-1;
    };
};
```

2.3.4 功能说明（函数、类）

```
//解决括号最长子串函数
void Longest(string input,Stack &s)
{//此处仍为伪代码 省略了部分具体的代码实现
    int maxlength=0;//最长子串长度
    int startindex=0;//最长子串起始位置
    s.data.push_back(-1);//以-1 作为边界,因为如果直接顺利匹配的话,栈内为空
    此时无法计算长度,需要借助一个手动输入的-1 来进行长度计算
    s.top++;
    for(int i=0;i<length;i++)
    {
        if(input[i]=='(')//算法思路:遇到左括号就让左括号的下标入栈
        {
            s.data[s.top + 1] = i;
            s.top++;
        }
        else if(input[i]==')')//遇到一个右括号时就让栈顶元素出栈,然后计算
当前的最长子串长度
        {
            if(s.top!=0)
            {
                s.data.pop_back();
                s.top--;
                int cur_length=i-s.data[s.top];
                if(cur_length>maxlength)
                {
                    maxlength=cur_length;
                    startindex=s.data[s.top]+1;
                }
                else if(cur_length==maxlength)
                {
                    if(s.data[s.top]+1<startindex)
                      startindex=s.data[s.top]+1;
                }
            }
            else//当遇到此时栈为空时,说明当前右括号没有匹配的左括号,将当前右
括号的下标入栈
            //即说明在此右括号之前没有合法的括号子串
            {
                s.data[s.top]=i;
            }
        }
    }
}
```

2.3.5 调试分析（遇到的问题和解决方法）

```
else
{
    s.data[s.top]=i;
}
```

在调试的过程中,遇到了一个困难,这个片段代码的意思是在当前栈为空时,当遇到右括号的时候,将右括号的下标作为字符串长度的左边界.
但是在起初的代码中,我的代码是在栈为空的时候,遇到右括号的时候,将长度边界重新恢复为-1,这样会导致任何情况下,字符串的长度都是当前下标减去(-1),这样会导致长度的计算错误

2.3.6 总结和体会

掌握了一种新的算法,通过栈记录左括号位置，能高效计算最长有效括号子串.

难点:如全为左括号、全为右括号、空串等情况需特殊处理。

易错点:当有多个最长子串时，需输出最靠前的起始位置，容易遗漏

2.4 队列的应用

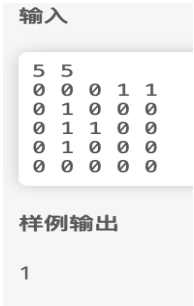
2.4.1 问题描述

简短地描述题目（可复制题目要求）
输入一个 n*m 的 0 1 矩阵，1 表示该位置有东西，0 表示该位置没有东西。所有四邻域联通的 1 算作一个区域，仅在矩阵边缘联通的不算作区域。

2.4.2 基本要求

输入:
第 1 行 2 个正整数 n，m，表示要输入的矩阵行数和列数
第 2—n+1 行为 n*m 的矩阵，每个元素的值为 0 或 1。
输出：1 行，代表区域数

Example:



2.4.3 数据结构设计

```
//BFS
void BFS(int x,int y,vector<vector<bool>>&grid)//使用一个二维数组来存储输入的矩阵
{
    int Row=grid.size();
    int Col=grid[0].size();
    queue<pair<int,int>>q;//使用一个队列来存储每一个点的相邻坐标
    q.push({x,y});
```

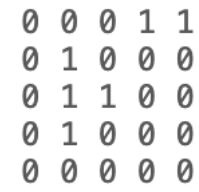
2.4.4 功能说明（函数、类）

```
void BFS(int x,int y,vector<vector<bool>>&grid)//使用一个二维数组来存储输入的矩阵
{
    int Row=grid.size();
    int Col=grid[0].size();
    queue<pair<int,int>>q;//使用一个队列来存储每一个点的相邻坐标
    q.push({x,y});
    grid[x][y]=0;//将此时被访问到的坐标标记为0
    while(!q.empty())
    {
        auto cur=q.front();//得到队列的首元素
        q.pop();
        for(int i=0;i<4;i++)
        {
            int tx;
            int ty;
            tx=cur.first+dx[i];
            ty=cur.second+dy[i];
            //如果新的坐标在矩阵范围内且值为1,则将其入队并将其值标记为0
            if(tx>=0 && tx<=Row-1 && ty>=0 && ty<=Col-1 &&
grid[tx][ty])
            {
                q.push({tx,ty});
                grid[tx][ty]=0;
            }
        }
    }
}
```

这段代码是解决这个问题的核心,通过 BFS 算法解决,在这段代码的下方还有一个 getIsLand(获取岛屿数量)的函数,在这个函数中遍历矩阵中的每一个坐标,每判断到一个 1,就将岛屿数量加 1，随后对该坐标调用 BFS 这个函数

2.4.5 调试分析（遇到的问题和解决方法）





这是题目里的一个特殊要求,仅在矩阵边缘联通的不算作区域,例如下图中右上角的两个联通的 1 就不算作区域,这与常规的 BFS 算法有一点区别.常规的 BFS 算法是遍历矩阵中的每一个点,但是如果这样做就不符合题意,容易把仅在边缘联通的矩阵算在内.经过思考后,我想到,只需要从遍历 M*N 的矩阵编程遍历(M-2)*(N-2)的矩阵即可,即不遍历第一行和第一列以及最后一行和最后一列,即可解决这个问题.

2.4.6 总结和体会

通过广度优先搜索（BFS）遍历岛屿，理解了队列在图遍历中的作用，提升了对图论基础算法的理解。

难点：循环时从 1 到 Row-1 和 Col-1，容易遗漏边界上的岛屿或导致越界。BFS 过程中新坐标的合法性判断（tx>=0 && tx<=Row-1 && ty>=0 && ty<=Col-1）必须严谨，否则会访问非法内存。

易错点:在边界处理时,忘记在 BFS 中将访问过的格子标记为 0，导致死循环或重复计数

算法效率：BFS 每个格子只访问一次，时间复杂度为 O(mn)，空间复杂度为 O(mn)，效率较高。但是 BFS 每个格子只访问一次，时间复杂度为 O(mn)，空间复杂度为 O(mn)，效率较高。

2.5 队列中的最大值问题

2.5.1 问题描述

给定一个队列，有下列 3 个基本操作：
（1）Enqueue(v): v 入队
（2）Dequeue(): 使队首元素删除，并返回此元素
（3）GetMax(): 返回队列中的最大元素
请设计一种数据结构和算法，让 GetMax 操作的时间复杂度尽可能地低。
要求运行时间不超过一秒

2.5.2 基本要求

输入：第 1 行 1 个正整数 n，表示队列的容量(队列中最多有 n 个元素)
接着读入多行，每一行执行一个动作。
若输入"dequeue"，表示出队，当队空时，输出一行“Queue is Empty”;否则，输出出队的元素;
若输入"enqueue m"，表示将元素 m 入队,当队满时(入队前队列中元素已有 n 个)，输出"Queue is Full"，否则，不输出;
若输入"max",输出队列中最大元素，若队空，输出一行“Queue is Empty”。
若输入"quit",结束输入，输出队列中的所有元素

输出：多行，分别是执行每次操作后的结果

2.5.3 数据结构设计

```
class max_queue
{
    private:
    vector<int>Queue;
    int front,rear;//front指向队列的头部，rear指向队列的尾部的下一个位置
    int size,capacity;
    deque<int>Max_queue;//用来存储队列中的最大值
```

2.5.4 功能说明（函数、类）

这里以 Enqueue 操作为例进行解释

```
void Enqueue(int m)
   {
       //Enqueue
       if(Is_full())
       {
           cout<<"Queue is Full"<<endl;
           return;
       }
//这里使用了循环队列,减少空间的浪费,因为 vector 在动态扩容时一次会增加多个空余位置
       Queue[rear]=m;
       rear=(rear+1) % capacity;
       size++;
       //更新最大值队列
       while(!Max_queue.empty() && Max_queue.back()<m)
       {
           Max_queue.pop_back();
       }
       Max_queue.push_back(m);
       return;
```

```
    }
```

类内存在有 Is_full Is_empty 两个函数,分别用来判断队列是否为空或者是否为满,便于 Enqueue 和 Dequeue 来进行使用

2.5.5 调试分析（遇到的问题和解决方法）

第一次写程序时,在处理输入命令时,没有考虑到大小写敏感的问题,把命令均写为了首字母大写,与题意不符,查了很久的代码逻辑,最后发现是命令的处理发生了错误,将处理输入命令的部分代码修改后逻辑就正确了.

2.5.6 总结和体会

结合循环队列和辅助队列，实现了高效的入队、出队和查询最大值操作.

难点: front 和 rear 的移动要用取模运算，容易写错导致队列逻辑混乱。

易错点:入队时要正确维护 Max_queue 的单调性，出队时要同步弹出最大值

算法效率: 所有操作均需 O(1) 时间复杂度，不能有多余遍历。

3.实验总结

通过本次的作业,我加深了对栈和队列的理解,写代码也更加的熟练了.最重要的是,在这几道题目的学习过程中,我还学习到了一些额外的算法,比如说求最长子串的算法,求岛屿联通经常使用的BFS(广度优先搜索算法),这些都是一些经典算法,我也去 LeetCode 上找了一些类似的算法题进行学习,这次实验学到的不仅是简单的数据结构,还学到了很多的算法知识,对我来说,这是一次成功的实验作业!